

■ Aprenda Go

Outline Completo do Curso

Ellen Korbes — github.com/vkorbes/aprendago

Curso completo de Go em português, cobrindo desde os fundamentos da linguagem até concorrência avançada, canais, tratamento de erros, testes e benchmarking. Estruturado em níveis ninja progressivos com exercícios práticos em cada etapa.

01 – Visão Geral do Curso

Bem-vindo!

- História do Go: criado por Ken Thompson (Unix, B, C), Rob Pike (UTF-8) e Robert Griesemer
- Em 2006, não existia linguagem com compilação rápida + execução eficiente + fácil de usar
- Killer feature: Thread = 1 MB | Goroutine = 2 KB
- Casos de uso: APIs, CLIs, microservices, processamento de dados, cloud e orquestração de containers

Por que Go?

- Eficiente: standard library poderosa, multiplataforma, garbage collection ultrarrápido
- Fácil de usar: pouquíssimas palavras reservadas, tipagem forte e estática
- Go é OOP? Sim e não — sem hierarquia de tipos, mas com métodos e interfaces

Sucesso & Recursos

- Foco é a chave — de gota em gota se enche um balde
- Slack: invite.slack.golangbridge.org | #brazil | #brazilian-go-studies
- Livros: A Linguagem de Programação Go (Donovan, Kernighan) | Go em Ação (Kennedy et al.)
- Documentação oficial: golang.org | godoc.org | Effective Go

02 – Variáveis, Valores & Tipos

Go Playground & Hello World

- Estrutura básica: package main, func main, import
- Pacotes: coleções de funções pré-prontas — notação pacote.Identificador

Operador Curto de Declaração (:=)

- Tipagem automática, só dentro de code blocks
- Diferente do operador de atribuição (=)
- Termin.: keywords, operadores, statement, expressão, scope

A palavra-chave var

- Para package-level scope
- Funciona em qualquer lugar do código

Explorando Tipos

- Tipos são estáticos em Go
- Primitivos: int, string, bool
- Compostos: slice, array, struct, map
- Composição: o ato de criar tipos compostos a partir de primitivos

Valor Zero & Pacote fmt

- Zeros: int=0, float=0.0, bool=false, string="", ponteiros/interfaces/slices/maps=nil
- fmt.Print / Println / Printf | fmt.Sprint / Sprintf | fmt.Fprint / Fprintf
- Format verbs: %v %T e outros

Criando Tipos & Conversão

- `type hotdog int` — tipos customizados com tipo subjacente
- Em Go: conversão (conversion), não casting
- `a = int(b)` — conversão explícita

03 – Exercícios: Ninja Nível 1

Exercícios Práticos

- Ex. 1: Operador curto de declaração com múltiplos tipos
- Ex. 2: Variáveis com `var` em package-level scope e valor zero
- Ex. 3: `fmt.Sprintf` para combinar valores em uma string
- Ex. 4 & 5: Criar tipo customizado e converter entre tipos

04 – Fundamentos da Programação

Tipo Booleano

- Binário: `true/false` — fundamental para lógica condicional
- Operadores relacionais: `==`, `<=`, `>=`, `!=`, `<`, `>` sempre retornam `bool`

Tipos Numéricos

- `int` & `uint` — 'implementation-specific sizes' | `byte = uint8` | `rune = int32`
- Regras gerais: use `int` para inteiros, `float64` para ponto flutuante
- Go é estática: `int` e `int32` são tipos distintos — conversão obrigatória
- Overflow: `uint16` vai de 0 a 65535 — ultrapassar gera comportamento inesperado

Tipo String

- Sequências de bytes, imutáveis
- Raw string literals vs. interpreted string literals
- Conversão para slice of bytes: `[]byte(x)`

Constantes & Iota

- Valores imutáveis, tipadas ou não-tipadas
- `iota`: identificador que representa números sequenciais em declarações de constantes
- Deslocamento de bits: `x << y` e `x >> y`

05 – Exercícios: Ninja Nível 2

Exercícios Práticos

- Ex. 1: Número em decimal, binário e hexadecimal
- Ex. 2: Expressões com operadores relacionais
- Ex. 3: Constantes tipadas e não-tipadas
- Ex. 4: Deslocamento de bits — demonstrar decimal, binário e hexadecimal

- Ex. 5: Raw string literal | Ex. 6: Constantes com iota para próximos 4 anos

06 – Fluxo de Controle

Tipos de Fluxo

- Sequencial: de cima para baixo
- Iterativo (loop): repetição com for
- Condicional: if/switch para tomada de decisões

Loops com for

- Sintaxe: inicialização, condição, pós — for i := 0; i < n; i++
- Loop condicional ('while'): for condition {}
- Loop infinito: for {} (usado em HTTP servers)
- Nested loop: relógio, calendário
- break e continue | operação módulo: %

Condicionais: if & switch

- if bool | if com declaração de inicialização | if / else if / else
- switch: sem fall-through por padrão | fall-through explícito | default | casos múltiplos
- switch com expressões, switch de tipo (type switch)
- Operadores lógicos: && || !

07 – Exercícios: Ninja Nível 3

Exercícios Práticos

- Ex. 1: Todos números de 1 a 10000
- Ex. 2: Unicode code points das letras maiúsculas do alfabeto
- Ex. 3 & 4: Loops com condição e loop infinito — anos desde seu nascimento
- Ex. 5: Resto da divisão | Ex. 6-9: if/else if/else e switch statements
- Ex. 10: Operadores lógicos && || !

08 – Agrupamentos de Dados

Array

- Tamanho fixo na declaração: var x [n]int | [5]int != [6]int
- Serve de fundação para slices — prefira slices no dia a dia

Slice

- Literal composta: x := []type{values}
- for range: for i, v := range x {}
- Fatiamento: x[:], x[a:], x[:b], x[a:b] — 'a' incluso, 'b' não
- append: x = append(slice, ...values) | x = append(slice, slice...)

- `make([]T, len, cap)`: otimiza alocação do array subjacente
- Slices multi-dimensionais: `[] []type`
- Atenção: a 'surpresa do array subjacente' ao deletar elementos

Maps

- Formato `key:value` — excelente performance para lookups
- `map[key]value{key: value}` | acesso: `m[key]`
- Comma ok idiom: `v, ok := m[key]`
- `range, delete(map, key)` — deletar key inexistente não gera erro

09 – Exercícios: Ninja Nível 4

Exercícios Práticos

- Ex. 1: Array de 5 ints com literal composta e `range`
- Ex. 2: Slice de 10 ints com `range` e `format printing`
- Ex. 3: Slicing — primeiro ao terceiro, quinto ao último, etc.
- Ex. 4 & 5: `append` de valores e slices | Ex. 6: `make` com estados do Brasil
- Ex. 7: Slice multi-dimensional `[] []string` com 3 pessoas
- Ex. 8-10: Map com key string e value `[]string` — adicionar e remover

10 – Structs

Struct

- Tipo de dados composto — armazena valores de tipos diferentes
- Declaração: `type x struct { y: z }` | acesso: `x.y`

Structs Embutidos & Anônimos

- Embedding: structs dentro de structs (composição)
- Exemplo: F1 driver é Pessoa (nome, idade) + Competidor (equipe, pontos)
- Structs anônimos: `x := struct { name type }{ name: value }`
- Campos promoted: acessíveis diretamente no tipo que embute

11 – Exercícios: Ninja Nível 5

Exercícios Práticos

- Ex. 1: Tipo 'pessoa' com nome, sobrenome e slice de sabores de sorvete
- Ex. 2: Map com sobrenomes como key e valores do tipo pessoa
- Ex. 3: Tipos veículo, caminhonete, sedan com embedding
- Ex. 4: Struct anônimo com map e slice internos

12 – Funções

Sintaxe & Conceitos

- `func (receiver) identifier(parameters) (returns) { code }`
- Parâmetros vs. argumentos | Tudo em Go é pass by value
- Funções variádicas: `parâmetro ...tipo`
- Múltiplos retornos: `func f() (int, string)`

Recursos Avançados

- Desenrolando slice: `funcao(slice...)` — `enumeration/unfurl`
- Defer: execução ao finalizar a função — muito usado para fechar arquivos
- Métodos: função anexada a um tipo — `valor.método()`

Interfaces & Polimorfismo

- Interface: permite que um valor tenha mais de um tipo
- Implementação implícita: se tem os métodos, implementa a interface
- `type x interface { método() tipo }`
- Type switch: `switch v.(type) { case Tipo1: ... }`

Funções como Valores

- Funções anônimas auto-executáveis: `func(p params){ ... }()`
- Func como expressão: `f := func(p params){ ... }`
- Retornando funções: `func f() func() int`
- Callback: passando função como argumento
- Closure: captura scope externo — isola dados entre chamadas
- Recursividade: função que chama a si mesma — ex: fatoriais

13 – Exercícios: Ninja Nível 6

Exercícios Práticos

- Ex. 1: Funções com um e com dois retornos
- Ex. 2: Parâmetro variádico — soma de ints
- Ex. 3: defer — demonstrar execução tardia
- Ex. 4: Tipo struct 'pessoa' com método que exibe nome completo e idade
- Ex. 5: Interface 'figura' com método área para quadrado e círculo
- Ex. 6-11: Função anônima, func como expressão, retornar função, callback, closure

14 – Ponteiros

O que são Ponteiros?

- Todo valor fica na memória — todo endereço tem uma localização
- `&variável` — mostra o endereço | `*variável` — mostra o valor no endereço (dereference)
- `*type` — tipo que contém endereço de um valor
- Exemplo: `a := 0; b := &a; *b++` — incrementa a através do ponteiro

Quando Usar Ponteiros

- Compartilhar endereços de memória — evitar cópias grandes
- Modificar o valor original (tudo em Go é pass by value!)
- Receivers de métodos que precisam modificar o struct

15 – Exercícios: Ninja Nível 7

Exercícios Práticos

- Ex. 1: Criar valor e demonstrar seu endereço na memória
- Ex. 2: Struct 'pessoa' com função `mudeMe(*pessoa)` — dereference e atalho `p1.nome`

16 – Aplicações

Standard Library na Prática

- Lendo documentação: golang.org/pkg | godoc.org — files, examples, funcs, types, methods
- JSON marshal: transformando structs em JSON — `json.Marshal()`
- JSON unmarshal: JSON para structs — `json.Unmarshal()` | tags de struct
- Marshal vs. Encoder: Marshal vai para variável | Encoder 'vai direto'

Interfaces Importantes

- `io.Writer`: type `Writer` interface { `Write(p []byte) (n int, err error)` }
- `fmt.Println` → `Fprintln(os.Stdout)` → `os.Stdout.Write()`
- `sort.Ints`, `sort.Strings` — altera o valor original!
- Sort customizado: implementar `sort.Interface` { `Len()`, `Less()`, `Swap()` }
- `bcrypt`: `GenerateFromPassword`, `CompareHashAndPassword`

17 – Exercícios: Ninja Nível 8

Exercícios Práticos

- Ex. 1: Marshal de `[]user` para JSON — atenção à pegadinha
- Ex. 2: Unmarshal com JSON-to-Go
- Ex. 3: `NewEncoder().Encode()` para Stdout — method chaining
- Ex. 4: `sort.Ints` e `sort.Strings`
- Ex. 5: Ordenar `[]user` por idade e sobrenome com sort customizado

18 – Concorrência

Goroutines & WaitGroups

- Go foi a primeira linguagem criada com multi-cores em mente (2007)
- Thread = 1 MB | Goroutine = 2 KB — múltiplas goroutines no mesmo endereço
- `go func()` — lança goroutine | `sync.WaitGroup`: `Add()`, `Done()`, `Wait()`
- `runtime.NumCPU()` | `runtime.NumGoroutine()`

Condição de Corrida (Race Condition)

- `go run -race main.go` — detecta race conditions
- Problema: duas goroutines lendo/escrevendo a mesma variável simultaneamente

Soluções: Mutex & Atomic

- Mutex: exclusão mútua — `sync.Mutex: Lock() / Unlock()`
- RWMutex: múltiplos leitores, um único escritor
- `atomic.AddInt64 / atomic.LoadInt64` — operações atômicas sem mutex

19 – Seu Ambiente de Desenvolvimento

Terminal & Go Workspace

- Shell commands: `pwd`, `ls`, `cd`, `touch`, `chmod`, `rm`, `cat`, `grep`, `nano`
- GOROOT: onde os binários do Go foram instalados
- GOPATH: seu workspace — `$HOME/go/bin` | `pkg` | `src`

Comandos Go

- `go run` | `go build` | `go install` | `go fmt` | `go vet` | `go doc`
- `go test -race` — detectar race conditions
- Compilação cruzada: `GOOS=darwin GOARCH=amd64 go build`

Pacotes & Exportação

- Uma pasta, vários arquivos — package scope compartilhado
- Exportado (visível): começa com Maiúscula | Não exportado: minúscula
- IDEs: VS Code + gopls | GoLand | Atom | Sublime

20 – Exercícios: Ninja Nível 9

Exercícios Práticos

- Ex. 1: 2 goroutines extra com `WaitGroups`
- Ex. 2: Conjuntos de métodos — interface com receiver ponteiro (*pessoa)
- Ex. 3: Programa incrementador com race condition (-race flag)
- Ex. 4 & 5: Corrigir race condition com Mutex e com Atomic
- Ex. 6: Demonstrar OS e ARCH — `go run`, `go build`, `go install`

21 – Canais (Channels)

Entendendo Canais

- Jeito Certo® de fazer sincronização e código concorrente
- `make(chan type, buffer)` — transmite valores entre goroutines
- Canais bloqueiam: como corredores de revezamento — 'passar o bastão'
- Send: `c <- 42` | Receive: `v := <-c`

Canais Direcionais

- Send channel: `chan<-` | Receive channel: `<-chan`
- Type safety: compilador impede operações erradas no canal
- Conversão: de geral para específico (bidirecional → unidirecional)

Padrões Avançados

- range e close: `for v := range canal` | `close(canal)`
- select: como switch para canais — escolhe um caso aleatório se vários estão prontos
- Comma ok idiom: `v, ok := <-chan`
- Convergência: vários canais → um canal | Divergência: um canal → vários goroutines
- Context: cancelamento de goroutines — `context.WithCancel`, `WithTimeout`, `WithDeadline`

22 – Exercícios: Ninja Nível 10

Exercícios Práticos

- Ex. 1: Função anônima auto-executável e buffer com canais
- Ex. 2-3: for range em canal | Ex. 4: select statement
- Ex. 5: Comma ok idiom | Ex. 6: 100 números em canal
- Ex. 7: 10 goroutines enviando 10 números cada para um canal

23 – Tratamento de Erros

Filosofia de Erros em Go

- Sem exceções (try/catch) — erros são valores como qualquer outro
- Múltiplos retornos facilitam reportar erros sem sobrecarregar o retorno
- package builtin: type error interface | package errors
- Regra: underscore em erros é pecado — sempre verifique!

Print, Log & Recover

- `fmt.Println()` → stdout sem timestamp
- `log.Println()` → timestamp + destino configurável
- `log.Fatalln()` → `os.Exit(1)` sem defer | `log.Panicln()` → panic (defer roda)
- `recover()` — recuperar de um panic dentro de defer

Erros Customizados

- `errors.New('mensagem')` | `fmt.Errorf('formato', args)`
- Criar tipo customizado que implementa a interface error
- 'Error values in Go aren't special' — Rob Pike

24 – Exercícios: Ninja Nível 11

Exercícios Práticos

- Ex. 1: Remover underscore e tratar erro adequadamente
- Ex. 2: Mensagem de erro customizada com `fmt.Errorf()`
- Ex. 3: Struct 'erroEspecial' implementando `builtin.error`
- Ex. 4: Usar struct como valor do tipo error
- Ex. 5: Desafio — descobrir como funcionam testes em Go por conta própria

25 – Documentação

Ferramentas de Documentação

- `go doc`: documentação na linha de comando — zero, um ou dois argumentos
- `godoc`: servidor local — `godoc -http=:8080 → localhost:8080`
- `godoc.org`: documentação pública — como publicar seu package

Escrevendo Documentação

- Comentário imediatamente antes da declaração — sem linhas em branco
- Começar com o nome do elemento: `'Println formats ...'`
- Para muito conteúdo: usar arquivo `doc.go` separado
- `gofmt` formata código | `go vet` detecta problemas | `golint` sugere melhorias

26 – Exercícios: Ninja Nível 12

Exercícios Práticos

- Ex. 1: Package 'cachorro' com função `Idade` documentada
- Ex. 2: Publicar no GitHub e ver documentação no `godoc.org`
- Ex. 3: `go doc` para `fmt`, `fmt Print`, `strings`, `strconv`

27 – Testes & Benchmarking

Testes em Go

- Arquivo terminando em `_test.go` na mesma package
- `func TestNome(t *testing.T) | go test | go test -v`
- Falhas: `t.Error()` com mensagem idiomática `'expected: x. got: y'`
- Testes em tabela: `[]struct` com data e answer esperado
- Testes como exemplos: `func ExampleFuncao()` com `// Output: resultado`

Ferramentas de Qualidade

- `gofmt`: formata o código automaticamente
- `go vet`: detecta constructs suspeitos (correctness)
- `golint`: sugere melhorias de estilo (suggestions)

Benchmark & Cobertura

- `func BenchmarkFunc(b *testing.B) { for i := 0; i < b.N; i++ { ... } }`

- `go test -bench .` (todos) | `go test -bench NomeFunc` (específico)
- `go test -cover` | `go test -coverprofile c.out`
- `go tool cover -html=c.out` — visualizar cobertura no browser
- Meta prática: 70-80% de cobertura é ótimo